

ValueTask in C# with Examples

In this article, I am going to discuss **ValueTask in C#** with Examples. Please read our previous article where we discussed [How to Attach Child Tasks to a Parent Task in C#](#) with Examples.

ValueTask in C#

Besides Task or Task<T>, another value that we can return from the asynchronous method is ValueTask or ValueTask<T>. The ValueTask or ValueTask<T> are relatively similar to Task or Task<T> with some crucial differences. ValueTask's mission is performance. The idea is to apply ValueTask in high-demand scenarios where there really is a measurable benefit. ValueTask is a struct. This implies that it is a value type, unlike Task which is a reference type.

The official documentation tells us that we can use ValueTask or ValueTask<T> in the following two conditions.

1. **Condition1:** When the result of the operation is most likely available synchronously.
2. **Condition2:** When the operation is used so frequently that the cost of using Task or Task<T> is significant.

Condition one relates to asynchronous methods that have the possibility of being executed synchronously. This is best visualized with an example. Please have a look at the following image.

Here, we have some code in which we have a dictionary outside of the method which acts as a cache layer so that when we look for a card with a certain Id, if that card available is on the dictionary, then we immediately return the value of the dictionary with the ID key. Now, if the value is not in the dictionary, then we can assume that we will go to a database. So, we will have to make an asynchronous operation. We will get the information on the card. We save that information in our Cache layer or dictionary and then we are going to return from our method.

```
public Dictionary<int, string> cardDictionary = new Dictionary<int, string>();
public async ValueTask<string> getCreditCard(int Id)
{
    if(cardDictionary.ContainsKey(Id))
    {
        //We return synchronously if we have the cards info in the dictionary
        return cardDictionary[Id];
    }

    //If not available in the dictionary, look for the card info in the database
    //asynchronous operation
    var card = "123456";
    cardDictionary[Id] = card;
    return await Task.FromResult(card);
}
```

As you can see this method, sometimes return synchronously when it returns from the dictionary, and other times it will return asynchronously when it has to go to the database for the information of the card. So, if we

know that this method is going to return synchronously almost always, then perhaps we should consider using ValueTask.

The other condition tells us about the performance and therefore it is a requirement that we measure. In order to improve something in terms of performance, we first need to measure it. Then from the data, we can make decisions and after making a change, we need to take another measurement and see whether the system has really improved or not. This means that unless you have done a previous measurement of the performance of the system, you should not introduce the use of ValueTask. So, in general, we should prefer the use of Task or Task<T>, and only if the performance analysis justifies it, you should switch to using ValueTask or ValueTask<T>.

Note: Please download **System.Threading.Tasks.Extensions** DLL from NuGet to work with ValueTask in C#.

Example to Understand ValueTask in C#:

```
using System;
using System.Threading.Tasks;
using System.Collections.Generic;
namespace AsynchronousProgramming
{
    class Program
    {
        public static Dictionary<int, string> cardDictionary = new Dictionary<
string>()
        {
            { 1001, "1001 Card Info" },
            { 1002, "1002 Card Info" },
            { 1003, "1003 Card Info" },
            { 1004, "1004 Card Info" }
        };
        static void Main(string[] args)
        {
            //Synchronous Call
            var Card1001Result = getCreditCard(1001);
            Console.WriteLine(Card1001Result);
            //Synchronous Call
            var Card1002Result = getCreditCard(1002);
            Console.WriteLine(Card1002Result);
            //Asynchronous Call
            var Card1006Result = getCreditCard(1006);
            Console.WriteLine(Card1006Result);
            Console.ReadKey();
        }
        public static async ValueTask<string> getCreditCard(int Id)
        {
            if (cardDictionary.ContainsKey(Id))
            {
                //We return synchronously if we have the cards info in the dic
                return cardDictionary[Id];
            }
        }
    }
}
```

```

        //If not available in the dictionary, look for the card info in the database
        //asynchronous operation
        var card = $"Card Info - {Id} From Database";
        cardDictionary[Id] = card;
        return await Task.FromResult(card);
    }
}

```

Output:

```

1001 Card Info
1002 Card Info
Card Info - 1006 From Database

```

Limitations of ValueTask in C#:

There are restrictions on the consumption of ValueTask. They are as follows:

1. The first one is that they cannot be Cache.
2. You cannot await it multiple times.
3. It does not support multiple continuations.
4. It is not thread-safe with an arbitrary number of threads capable of concurrently registering continuations.
5. They do not support a blocking model where the asynchronous method can block the current thread instead of releasing it using await operator.

When to use ValueTask in C#?

It is recommended to use Task or Task<T> in general and only use ValueTask and ValueTask<T> when performance measurements justify it, which may be the case in High-performance scenarios.

In the next article, I am going to discuss the [Canceling Non-Cancellable Tasks with TaskCompletionSource in C#](#) with Examples. Here, in this article, I try to explain **Value Task in C#** with Examples. I hope you enjoy this Value Task in C# with Examples article.